Graphs using Java

Chapter 1 – Introduction to Graphs

Table of Contents

---

1. Introduction

A **Graph** is a non-linear data structure consisting of a set of **vertices (nodes)** connected by **edges**.

Unlike trees, graphs do not have restrictions such as:

- One root
- Parent-child relationship

- No cycles

Graphs are among the most important data structures because they model relationships.

Examples include:

- Facebook friends
- Instagram followers
- Road Maps
- Flight Networks
- Computer Networks
- Dependency Graphs
- Recommendation Systems
- Web Pages connected using hyperlinks

Formal Definition

A graph is represented as

G = (V, E)

Where

- **V** = Set of Vertices
- **E** = Set of Edges

Example

```
V = {A, B, C, D}

E = {
(A,B),
(B,C),
(C,D),
(A,D)
}
```

Visual Representation

```
    A
   / \
  B---D
```

```
    \
    C
```

Here

Vertices = 4

Edges = 4

Why Study Graphs?

Many real-world problems are naturally represented as graphs.

Examples

Problem	Graph Representation
Google Maps	Cities = Vertices, Roads = Edges
Facebook	Users = Vertices, Friendships = Edges
LinkedIn	Professionals = Vertices
Airline Network	Airports connected by Flights
Internet	Routers connected using cables
Electric Grid	Power Stations connected through transmission lines
Compiler	Dependency Graph
Course Scheduling	Prerequisite Graph

---

2. Real World Example

Imagine four cities.

```
Bangalore
Mysore
```

```
Chennai
Hyderabad
```

Roads

```
Bangalore <-> Mysore

Bangalore <-> Chennai

Chennai <-> Hyderabad

Mysore <-> Hyderabad
```

Graph

```
        Bangalore
        /       \
   Mysore       Chennai
      \         /
       Hyderabad
```

Finding

- Shortest Route
- Minimum Cost
- Alternate Path

all become Graph Problems.

Social Network Example

```
Ashwin ----- Rahul
   |           |
   |           |
 Keerthi ---- Arjun
```

Questions

Who are Ashwin's friends?

How many mutual friends exist?

Is Rahul connected to Arjun?

Can Ashwin reach Arjun?

These are graph traversal problems.

---

3. Terminology

Vertex

A node in the graph.

```
A
B
C
D
```

Total Vertices = 4

Edge

Connection between two vertices.

```
A ----- B
```

is one edge.

Adjacent Vertices

Vertices connected directly.

```
A ----- B
```

A and B are adjacent.

Incident Edge

An edge touching a vertex.

```
A ---- B
```

Edge (A,B) is incident on A and B.

Neighbor

Immediate connected vertex.

```
A

/ \

B C
```

Neighbors of A

```
B
C
```

4. Types of Graphs

Graphs can be classified in many ways.

Undirected Graph

Edges have no direction.

```
A ----- B
```

Means

```
A can reach B

B can reach A
```

Road Example

```
Two-way Road
```

Diagram

```
    A
   / \
  B---C
```

Directed Graph

Edges have direction.

```
A ----> B
```

Means

```
A reaches B

B cannot reach A
```

Diagram

```
A ---> B ---> C
```

Used in

- Twitter Follow
- Course Dependency
- Workflow
- Build Systems

Difference

Undirected	Directed
Bidirectional	One Direction
Two-way Road	One-way Road
Facebook Friend	Twitter Follow

---

5. Weighted Graph

Edges store weights.

```
A --5-- B

B --2-- C

A --7-- C
```

Example

Weight = Distance

Weight = Cost

Weight = Time

Weight = Fuel

Example

```
Bangalore --140--> Mysore

Bangalore --350--> Chennai
```

Need shortest path.

Algorithms

- Dijkstra
- Bellman Ford
- Floyd Warshall

---

6. Unweighted Graph

All edges treated equally.

```
A

/ \

B C
```

Distance

Every edge = 1

Used in

- BFS
- Social Networks
- Game Maps

---

7. Connected Graph

Every vertex is reachable.

```
   A
  / \
 B   C
  \ /
   D
```

Every node can reach every other node.

---

8. Disconnected Graph

```
A ---- B

C ---- D
```

Two separate components.

Traversal must start from every unvisited vertex.

---

9. Complete Graph

Every vertex connects to every other vertex.

Example

```
A

/|\

B-C

\|/

D
```

Formula

Number of edges

```
n(n−1)/2
```

Example

4 vertices

```
4×3/2 = 6 edges
```

---

10. Cyclic Graph

Contains a cycle.

```
A

/ \

B--C
```

Cycle

```
A → B → C → A
```

---

11. Acyclic Graph

No cycles.

Example

```
A

|

B

|

C

|

D
```

Trees are acyclic graphs.

---

12. Tree as a Graph

A tree is a special graph.

Properties

- Connected
- No cycles
- n-1 edges

Example

```
    1

  /   \

 2     3

/ \

4   5
```

13. Sparse Graph

Very few edges.

```
10 vertices

12 edges
```

Representation

Adjacency List

preferred.

Dense Graph

Almost every possible edge exists.

```
100 vertices

4900 edges
```

Representation

Adjacency Matrix

preferred.

---

14. Degree of Vertex

Number of edges connected.

Example

```
    A
   / \
  B   C
   \
    D
```

Degree

```
A = 2

B = 2

C = 1

D = 1
```

Directed Graph Degree

Two types.

In Degree

Incoming edges.

Out Degree

Outgoing edges.

Example

```
A ---> B ---> C
       ^
       |
       D
```

Vertex B

```
In Degree = 2

Out Degree = 1
```

---

15. Path

A sequence of connected vertices.

```
A → B → C → D
```

Length

```
3 edges
```

Simple Path

No repeated vertices.

```
A
↓
B
↓
C
↓
D
```

16. Cycle

Starts and ends at same vertex.

```
A
↓
B
↓
C
↓
A
```

Length

```
3
```

17. Connected Components

Example

```
A ----- B


C ----- D ----- E


F
```

Components

```
{A,B}

{C,D,E}

{F}
```

Total Components

```
3
```

---

18. Graph Representation (Preview)

There are two common methods.

Adjacency Matrix

```
0 1 0 1

1 0 1 0

0 1 0 1

1 0 1 0
```

Fast edge lookup.

Consumes more memory.

Adjacency List

```
0 -> 1 -> 3

1 -> 0 -> 2

2 -> 1 -> 3

3 -> 0 -> 2
```

Memory efficient.

Most commonly used in coding interviews.

Detailed implementation will be covered in **Chapter 2**.

---

19. Complexity Summary

Operation	Matrix	List
Space	O(V²)	O(V + E)
Add Edge	O(1)	O(1)
Remove Edge	O(1)	O(V)
Check Edge	O(1)	O(Degree)
Traverse Neighbors	O(V)	O(Degree)

---

20. Common Interview Questions

1. Difference between Tree and Graph?
2. Why is BFS implemented using a Queue?
3. Why does DFS use a Stack or Recursion?
4. When should you use an Adjacency Matrix?

5. When should you use an Adjacency List?
6. Can a Graph have multiple cycles?
7. Can a Tree be considered a Graph?
8. What is the maximum number of edges in an undirected graph?
9. What is a Complete Graph?
10. Explain Connected Components.

---

21. Practice Questions

Basic

1. Define a graph.
2. Explain the difference between vertex and edge.
3. What is an adjacency vertex?
4. Explain weighted and unweighted graphs.
5. What is a connected graph?

Intermediate

6. Draw an undirected graph with 6 vertices.
7. Draw a directed graph containing one cycle.
8. Find the degree of every vertex in a given graph.
9. Identify connected components.
10. Calculate the maximum edges possible for 8 vertices.

Advanced

11. Explain why trees are special graphs.
12. Compare sparse and dense graphs.
13. Explain the difference between a path and a cycle.
14. Explain why an adjacency list is preferred for sparse graphs.
15. Give five real-world applications of graphs.

Key Takeaways

- A graph models relationships between entities.
- Graphs consist of **vertices** and **edges**.
- Graphs may be directed or undirected.
- Graphs may be weighted or unweighted.

- Trees are a special type of graph.
- Connected components represent isolated groups of vertices.
- The two primary graph representations are **Adjacency Matrix** and **Adjacency List**.
- Graph traversal algorithms such as **BFS** and **DFS** build upon these representations.

Chapter 2 – Graph Representation

Learning Objectives

After completing this chapter, you will be able to:

- Understand why graph representation is important.
- Represent graphs using an Adjacency Matrix.
- Represent graphs using an Adjacency List.
- Implement both representations in Java.
- Compare the advantages and disadvantages of each representation.
- Determine which representation is suitable for different problems.

---

1. Why Do We Need Graph Representation?

A graph consists of vertices and edges, but simply knowing the vertices and edges is not enough. A computer must store this information in memory so that graph algorithms such as BFS, DFS, Dijkstra's Algorithm, and Topological Sort can process it efficiently.

The choice of representation directly affects:

- Memory usage
- Speed of edge lookup
- Speed of traversal
- Ease of implementation

The two most commonly used graph representations are:

1. Adjacency Matrix
2. Adjacency List

Example Graph

Consider the following undirected graph.

```
      0
    / | \
   1  2  3
      |
      4
```

Edges

```
(0,1)
(0,2)
(0,3)
(2,4)
```

Number of vertices

```
V = 5
```

Number of edges

```
E = 4
```

---

2. Adjacency Matrix

Definition

An Adjacency Matrix is a two-dimensional array of size

```
V × V
```

where

```
matrix[i][j]
```

stores whether an edge exists between vertex i and vertex j.

For an unweighted graph,

```
1 → Edge exists

0 → No edge
```

Matrix for the Example Graph

```
     0 1 2 3 4
    ---------------
0 |  0 1 1 1 0
1 |  1 0 0 0 0
2 |  1 0 0 0 1
3 |  1 0 0 0 0
4 |  0 0 1 0 0
```

Visual Relationship

```
Edge (0,1)

matrix[0][1] = 1
matrix[1][0] = 1
```

Since the graph is undirected,

```
matrix[i][j] == matrix[j][i]
```

The matrix is symmetric.

Memory Layout

```
      Columns
```

```
     0 1 2 3 4

Rows

0    0 1 1 1 0

1    1 0 0 0 0

2    1 0 0 0 1

3    1 0 0 0 0

4    0 0 1 0 0
```

Rows represent the source vertex.

Columns represent the destination vertex.

Java Implementation

```java
public class GraphMatrix {

    private int vertices;
    private int[][] matrix;

    public GraphMatrix(int vertices) {
        this.vertices = vertices;
        matrix = new int[vertices][vertices];
    }

    public void addEdge(int source, int destination) {
        matrix[source][destination] = 1;
        matrix[destination][source] = 1;
    }

    public void printMatrix() {

        System.out.print("    ");

        for (int i = 0; i < vertices; i++) {
            System.out.print(i + " ");
        }

        System.out.println();
```

```
        for (int i = 0; i < vertices; i++) {

            System.out.print(i + " : ");

            for (int j = 0; j < vertices; j++) {
                System.out.print(matrix[i][j] + " ");
            }

            System.out.println();
        }
    }

    public static void main(String[] args) {

        GraphMatrix graph = new GraphMatrix(5);

        graph.addEdge(0,1);
        graph.addEdge(0,2);
        graph.addEdge(0,3);
        graph.addEdge(2,4);

        graph.printMatrix();
    }
}
```

Output

```
    0 1 2 3 4

0 : 0 1 1 1 0
1 : 1 0 0 0 0
2 : 1 0 0 0 1
3 : 1 0 0 0 0
4 : 0 0 1 0 0
```

Adding an Edge

Suppose we add

```
(3,4)
```

The matrix changes as follows.

Before

```
3 : 1 0 0 0 0
4 : 0 0 1 0 0
```

After

```
3 : 1 0 0 0 1
4 : 0 0 1 1 0
```

Removing an Edge

Removing

```
(0,3)
```

```
public void removeEdge(int source, int destination) {

    matrix[source][destination] = 0;
    matrix[destination][source] = 0;

}
```

Checking Whether an Edge Exists

```
public boolean hasEdge(int source, int destination) {

    return matrix[source][destination] == 1;

}
```

Example

```
hasEdge(0,2)
```

Output

```
true
```

Degree of a Vertex

The degree equals the number of ones in its row.

Example

```
0 : 0 1 1 1 0
```

Degree of vertex 0

```
3
```

Java

```
public int degree(int vertex) {

    int count = 0;

    for (int i = 0; i < vertices; i++) {

        if (matrix[vertex][i] == 1)
            count++;

    }

    return count;
}
```

Advantages of Adjacency Matrix

- Simple implementation
- Fast edge lookup
- Easy to understand
- Good for dense graphs
- Easy to modify edges

Disadvantages

- High memory usage
- Inefficient for sparse graphs
- Traversing neighbors requires scanning the entire row

Time Complexity

Operation	Complexity
Add Edge	O(1)
Remove Edge	O(1)
Check Edge	O(1)
Traverse Neighbors	O(V)
Space	O(V²)

3. Adjacency List

Definition

An Adjacency List stores, for every vertex, a list containing all of its neighboring vertices.

Instead of storing every possible edge, only the existing edges are stored.

Example

```
      |
      4
```

Adjacency List

```
0 → 1 → 2 → 3

1 → 0

2 → 0 → 4

3 → 0

4 → 2
```

Visualization

```
Array

0

1

2

3

4
```

Each array element points to a list.

```
0 → 1 → 2 → 3

1 → 0

2 → 0 → 4

3 → 0
```

```
4 → 2
```

Java Representation

The most common representation uses

```
ArrayList<ArrayList<Integer>>
```

Each index stores its neighbors.

Java Implementation

```
import java.util.ArrayList;

public class GraphList {

    private int vertices;
    private ArrayList<ArrayList<Integer>> adjList;

    public GraphList(int vertices) {

        this.vertices = vertices;

        adjList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<Integer>());
        }
    }

    public void addEdge(int source, int destination) {

        adjList.get(source).add(destination);
        adjList.get(destination).add(source);

    }

    public void printGraph() {

        for (int i = 0; i < vertices; i++) {
```

```
            System.out.print(i + " -> ");

            for (int neighbor : adjList.get(i)) {
                System.out.print(neighbor + " ");
            }

            System.out.println();
        }
    }

    public static void main(String[] args) {

        GraphList graph = new GraphList(5);

        graph.addEdge(0,1);
        graph.addEdge(0,2);
        graph.addEdge(0,3);
        graph.addEdge(2,4);

        graph.printGraph();
    }
}
```

Output

```
0 -> 1 2 3

1 -> 0

2 -> 0 4

3 -> 0

4 -> 2
```

Removing an Edge

```
public void removeEdge(int source, int destination) {

    adjList.get(source).remove(Integer.valueOf(destination));
    adjList.get(destination).remove(Integer.valueOf(source));
```

```
}
```

Checking Whether an Edge Exists

```
public boolean hasEdge(int source, int destination) {

    return adjList.get(source).contains(destination);

}
```

Degree of a Vertex

The degree equals the size of its adjacency list.

```
public int degree(int vertex) {

    return adjList.get(vertex).size();

}
```

Traversing Neighbors

```
for (int neighbor : adjList.get(0)) {

    System.out.println(neighbor);

}
```

Output

```
1

2

3
```

Memory Comparison

Suppose a graph has

```
1000 vertices

1500 edges
```

Adjacency Matrix

```
1000 × 1000

1,000,000 entries
```

Adjacency List

Only

```
1000 lists

3000 stored edge values
```

because each undirected edge is stored twice.

The memory savings become significant for sparse graphs.

Weighted Graph Representation

Each edge stores both the destination vertex and its weight.

Example

```
0 --5-- 1

0 --2-- 2

2 --7-- 4
```

One common Java representation is:

```
class Edge {

    int destination;
    int weight;

    Edge(int destination, int weight) {
        this.destination = destination;
        this.weight = weight;
    }
}
```

Adjacency List

```
ArrayList<ArrayList<Edge>>
```

Example

```
0 → (1,5) → (2,2)

1 → (0,5)

2 → (0,2) → (4,7)

3

4 → (2,7)
```

Directed Graph Representation

Only one direction is stored.

Example

```
0 → 1

0 → 2

2 → 4
```

Adjacency List

```
0 → 1 → 2

1

2 → 4

3

4
```

Java

```
public void addEdge(int source, int destination) {

    adjList.get(source).add(destination);

}
```

Notice that the reverse edge is not added.

Matrix vs List

Feature	Adjacency Matrix	Adjacency List
Storage	2D Array	Array of Lists
Space Complexity	O(V²)	O(V + E)
Edge Lookup	O(1)	O(Degree)
Add Edge	O(1)	O(1)
Remove Edge	O(1)	O(Degree)
Neighbor Traversal	O(V)	O(Degree)
Dense Graph	Excellent	Good

Feature	Adjacency Matrix	Adjacency List
Sparse Graph	Poor	Excellent
Memory Usage	High	Low
Most Common in Interviews	Rare	Very Common

Choosing the Right Representation

Situation	Recommended Representation
Social Networks	Adjacency List
Road Networks	Adjacency List
Computer Networks	Adjacency List
Dense Graph Algorithms	Adjacency Matrix
Competitive Programming	Adjacency List
BFS	Adjacency List
DFS	Adjacency List
Dijkstra's Algorithm	Adjacency List
Topological Sort	Adjacency List

Common Mistakes

1. Forgetting to add the reverse edge in an undirected graph.
2. Adding the reverse edge in a directed graph.
3. Using an adjacency matrix for large sparse graphs, resulting in excessive memory usage.
4. Removing an element from an `ArrayList<Integer>` using the index instead of the value.

5. Confusing vertex indices with edge weights in weighted graph representations.

Practice Questions

1. Implement an adjacency matrix for a graph with six vertices.
2. Add and remove edges in an adjacency matrix.
3. Find the degree of every vertex using an adjacency matrix.
4. Implement an adjacency list for an undirected graph.
5. Modify the adjacency list implementation for a directed graph.
6. Implement a weighted graph using an `Edge` class.
7. Write a method to determine whether an edge exists between two vertices.
8. Compare the memory usage of an adjacency matrix and an adjacency list for a graph with 1000 vertices and 1500 edges.
9. Print all neighbors of a given vertex.
10. Explain why adjacency lists are generally preferred in graph algorithms such as BFS and DFS.

Chapter 3 – Breadth First Search (BFS)

Learning Objectives

After completing this chapter, you will be able to:

- Understand Breadth First Search (BFS).
- Learn how BFS traverses a graph.
- Understand why a Queue is used.
- Implement BFS using Java.
- Perform dry runs of the algorithm.
- Analyze time and space complexity.
- Understand applications of BFS.
- Solve common interview questions based on BFS.

---

1. Introduction to Breadth First Search

Breadth First Search (BFS) is a graph traversal algorithm that explores a graph **level by level**.

Instead of moving as deep as possible, BFS visits all the neighboring vertices before moving to the next level.

It is one of the fundamental graph traversal algorithms and forms the basis for many advanced graph algorithms.

Real-Life Analogy

Imagine dropping a stone into a pond.

The ripples spread outward in concentric circles.

```
        0
     0     0
   0         0
 0             0
```

Similarly, BFS starts from one vertex and explores all vertices at the current distance before moving farther away.

Example Graph

```
        0
      /  |  \
     1   2   3
    / \      |
   4   5     6
```

If BFS starts from vertex **0**, the traversal order is

```
0 1 2 3 4 5 6
```

Notice that every level is completely explored before the next level begins.

Level-wise Traversal

```
Level 0

      0
```

```
Level 1

    1   2   3
```

```
Level 2

    4   5   6
```

Traversal

```
0

↓

1 2 3

↓

4 5 6
```

Why Does BFS Use a Queue?

A Queue follows the **First In First Out (FIFO)** principle.

The first discovered vertex is processed first.

```
Insert

1

2
```

```
3

Remove

1

↓

2

↓

3
```

Because neighbors are processed in the same order in which they are discovered, BFS naturally performs level-order traversal.

BFS Algorithm

Suppose we start from vertex **0**.

Step 1

Visit vertex 0.

```
Visited

0
```

Queue

```
0
```

Step 2

Remove 0.

Visit all neighbors.

```
1
2
3
```

Queue

```
1 2 3
```

Step 3

Remove 1.

Visit its unvisited neighbors.

```
4
5
```

Queue

```
2 3 4 5
```

Step 4

Remove 2.

No new neighbors.

Queue

```
3 4 5
```

Step 5

Remove 3.

Visit

```
6
```

Queue

```
4 5 6
```

Step 6

Remove 4.

Queue

```
5 6
```

Step 7

Remove 5.

Queue

```
6
```

Step 8

Remove 6.

Queue becomes empty.

Traversal ends.

BFS Pseudocode

```
Mark source vertex as visited
```

```
Insert source into Queue

While Queue is not empty

    Remove front vertex

    Process it

    For every neighbor

        If not visited

            Mark visited

            Insert into Queue
```

Java Implementation

```java
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;

public class GraphBFS {

    private int vertices;
    private ArrayList<ArrayList<Integer>> adjList;

    public GraphBFS(int vertices) {

        this.vertices = vertices;
        adjList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<Integer>());
        }
    }

    public void addEdge(int source, int destination) {

        adjList.get(source).add(destination);
        adjList.get(destination).add(source);

    }
```

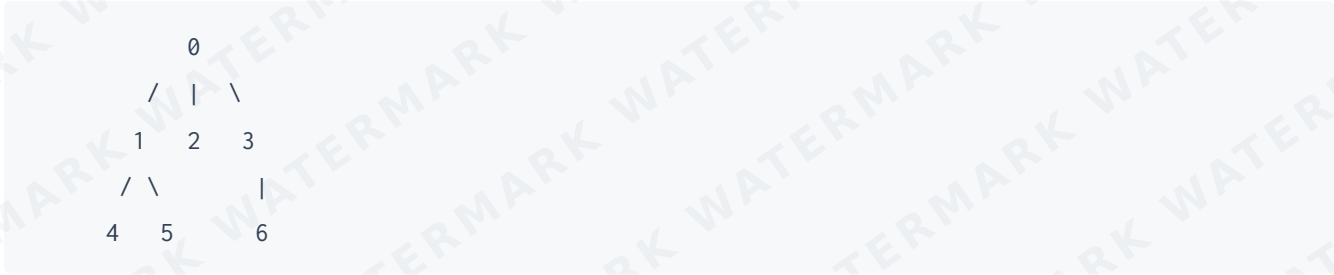
```
    public void bfs(int start) {
        boolean[] visited = new boolean[vertices];
        Queue<Integer> queue = new LinkedList<>();
        visited[start] = true;
        queue.offer(start);
        while (!queue.isEmpty()) {
            int current = queue.poll();
            System.out.print(current + " ");
            for (int neighbor : adjList.get(current)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }
    }
    public static void main(String[] args) {
        GraphBFS graph = new GraphBFS(7);
        graph.addEdge(0,1);
        graph.addEdge(0,2);
        graph.addEdge(0,3);
        graph.addEdge(1,4);
        graph.addEdge(1,5);
        graph.addEdge(3,6);
        graph.bfs(0);
    }
}
```

Output

```
0 1 2 3 4 5 6
```

Dry Run

Graph



Initial State

Queue

```
0
```

Visited

```
0
```

Remove

```
0
```

Visit

```
1

2
```

```
3
```

Queue

```
1 2 3
```

Visited

```
0 1 2 3
```

Remove

```
1
```

Visit

```
4

5
```

Queue

```
2 3 4 5
```

Visited

```
0 1 2 3 4 5
```

Remove

```
2
```

Queue

```
3 4 5
```

Remove

```
3
```

Visit

```
6
```

Queue

```
4 5 6
```

Remove

```
4
```

Queue

```
5 6
```

Remove

```
5
```

Queue

```
6
```

Remove

```
6
```

Queue

```
Empty
```

Traversal

```
0 1 2 3 4 5 6
```

BFS on a Disconnected Graph

A single BFS only visits the connected component containing the starting vertex.

Example

```
        0 ---- 1

        2 ---- 3 ---- 4

        5
```

Starting from **0**

Traversal

```
0 1
```

Vertices

```
2 3 4 5
```

remain unvisited.

Complete BFS Traversal

To traverse an entire disconnected graph, start BFS from every unvisited vertex.

```
public void bfsTraversal() {

    boolean[] visited = new boolean[vertices];

    for (int i = 0; i < vertices; i++) {
```

```
        if (!visited[i]) {
            Queue<Integer> queue = new LinkedList<>();
            visited[i] = true;
            queue.offer(i);
            while (!queue.isEmpty()) {
                int current = queue.poll();
                System.out.print(current + " ");
                for (int neighbor : adjList.get(current)) {
                    if (!visited[neighbor]) {
                        visited[neighbor] = true;
                        queue.offer(neighbor);
                    }
                }
            }
        }
    }
}
```

Output

```
0 1 2 3 4 5
```

BFS Using an Adjacency Matrix

```
public void bfsMatrix(int[][] graph, int start) {
    boolean[] visited = new boolean[graph.length];
    Queue<Integer> queue = new LinkedList<>();
```

```
        visited[start] = true;
        queue.offer(start);
        while (!queue.isEmpty()) {
            int current = queue.poll();
            System.out.print(current + " ");
            for (int i = 0; i < graph.length; i++) {
                if (graph[current][i] == 1 && !visited[i]) {
                    visited[i] = true;
                    queue.offer(i);
                }
            }
        }
    }
```

Time Complexity

Let

```
V = Number of Vertices

E = Number of Edges
```

Using an Adjacency List

Operation	Complexity
Visit Vertices	O(V)
Visit Edges	O(E)

Operation	Complexity
Total	**O(V + E)**

Using an Adjacency Matrix

Every row contains **V** columns.

Operation	Complexity
Traversal	**O(V²)**

Space Complexity

Visited Array

```
O(V)
```

Queue

```
O(V)
```

Total

```
O(V)
```

Applications of BFS

- Shortest path in an unweighted graph
- Level-order traversal of a tree
- Finding connected components
- Social network friend suggestions
- GPS navigation with equal edge weights
- Web crawling

- Network broadcasting
- Peer-to-peer communication
- Flood fill algorithm
- Minimum number of moves in puzzles

BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal	Level by Level	Depth First
Shortest Path (Unweighted)	Yes	No
Memory Usage	Higher	Lower (usually)
Recursive	No	Yes (commonly)
Used In	Shortest Path, Connected Components	Cycle Detection, Topological Sort

---

Common Mistakes

1. Forgetting to mark a vertex as visited **before** adding it to the queue.
2. Marking vertices as visited only after removing them from the queue, causing duplicate insertions.
3. Forgetting to initialize the visited array.
4. Assuming one BFS covers a disconnected graph.
5. Using a Stack instead of a Queue.
6. Omitting the visited check before enqueuing neighbors.

Interview Questions

1. Why does BFS use a Queue?
2. Why is BFS suitable for finding the shortest path in an unweighted graph?
3. What happens if the graph contains a cycle?
4. Why do we need a visited array?
5. Can BFS be implemented recursively?
6. How do you perform BFS on a disconnected graph?
7. Compare BFS using an adjacency matrix and an adjacency list.
8. Explain the time complexity of BFS.

Practice Questions

1. Implement BFS using an adjacency list.
2. Implement BFS using an adjacency matrix.
3. Print the vertices level by level.
4. Count the number of vertices reachable from a given source.
5. Perform BFS on a disconnected graph.
6. Find the shortest distance from a source vertex in an unweighted graph.
7. Count the connected components using BFS.
8. Print the traversal order for a given graph.
9. Determine whether a path exists between two vertices using BFS.
10. Modify BFS to store the parent of every visited vertex.

LeetCode and GeeksforGeeks Practice

Problem	Platform
Number of Islands (200)	LeetCode
Rotting Oranges (994)	LeetCode
Flood Fill (733)	LeetCode
Clone Graph (133)	LeetCode
Open the Lock (752)	LeetCode
Word Ladder (127)	LeetCode

Problem	Platform
Shortest Path in Binary Matrix (1091)	LeetCode
BFS of Graph	GeeksforGeeks
Level Order Traversal	GeeksforGeeks
Connected Components	GeeksforGeeks

Key Points

- Breadth First Search visits vertices **level by level**.
- BFS always uses a **Queue (FIFO)**.
- Each vertex is visited exactly once.
- In an adjacency list representation, BFS runs in **O(V + E)** time.
- In an adjacency matrix representation, BFS runs in **O(V²)** time.
- BFS guarantees the shortest path in an **unweighted graph**.
- For disconnected graphs, BFS must be started from every unvisited vertex.

Chapter 4 – Depth First Search (DFS)

Learning Objectives

After completing this chapter, you will be able to:

- Understand the concept of Depth First Search (DFS).
- Learn why DFS uses recursion or a stack.
- Implement DFS using recursion.
- Implement DFS using an explicit stack.
- Perform a step-by-step dry run.
- Traverse disconnected graphs using DFS.
- Analyze time and space complexity.
- Understand real-world applications of DFS.

---

1. Introduction to Depth First Search

Depth First Search (DFS) is a graph traversal algorithm that explores a graph by going **as deep as possible** along a path before backtracking.

Unlike Breadth First Search, which visits vertices level by level, DFS follows one branch until no further progress is possible and then returns to explore other branches.

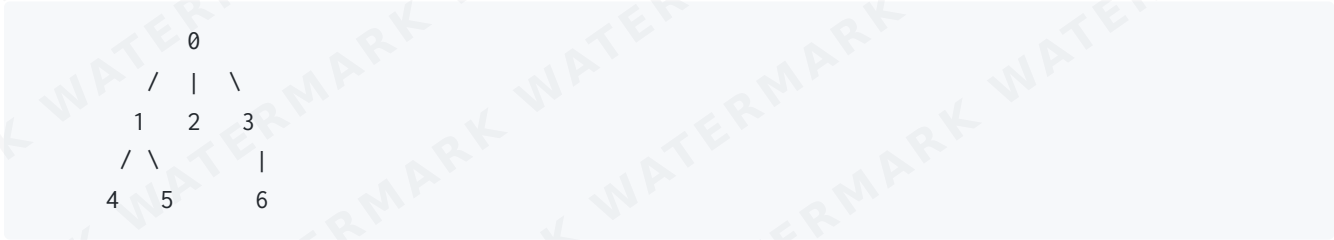
Real-Life Analogy

Imagine exploring a maze.

Instead of checking every nearby path, you choose one path and continue until you reach a dead end. Then you return to the previous junction and explore another unexplored path.

This is exactly how DFS works.

Example Graph



One possible DFS traversal starting from vertex **0** is

```
0 1 4 5 2 3 6
```

> **Note:** DFS traversal is **not unique**. The order depends on the order in which neighbors are stored.

DFS Traversal

```
Start

0

↓

1

↓

4

↑

↓

5

↑

↑

↓

2

↑

↓

3

↓

6
```

Why Does DFS Use a Stack?

DFS always remembers the most recently visited vertex so that it can continue exploring deeper.

This behavior follows the **Last In First Out (LIFO)** principle.

A **Stack** naturally provides this behavior.

Recursion internally uses the program's call stack, which is why recursive DFS works without explicitly creating a stack.

DFS Algorithm

1. Visit the current vertex.
2. Mark it as visited.
3. Visit the first unvisited neighbor.
4. Continue recursively.
5. If no unvisited neighbors remain, backtrack.
6. Continue until all reachable vertices are visited.

DFS Pseudocode

```
DFS(vertex)

Mark vertex as visited

Process vertex

For every neighbor

    If neighbor is not visited

        DFS(neighbor)
```

Recursive DFS Visualization

Graph

Call Stack

```
DFS(0)
↓
DFS(1)
↓
DFS(4)
↓
Return
↓
DFS(5)
↓
Return
↓
Return
↓
DFS(2)
↓
Return
↓
DFS(3)
↓
DFS(6)
↓
Return
↓
Return
```

Java Implementation (Recursive DFS)

```
import java.util.ArrayList;

public class GraphDFS {

    private int vertices;
    private ArrayList<ArrayList<Integer>> adjList;

    public GraphDFS(int vertices) {

        this.vertices = vertices;
        adjList = new ArrayList<>();

        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<Integer>());
        }
    }

    public void addEdge(int source, int destination) {

        adjList.get(source).add(destination);
        adjList.get(destination).add(source);

    }

    public void dfs(int start) {

        boolean[] visited = new boolean[vertices];

        dfsUtil(start, visited);

    }

    private void dfsUtil(int current, boolean[] visited) {

        visited[current] = true;

        System.out.print(current + " ");

        for (int neighbor : adjList.get(current)) {

            if (!visited[neighbor]) {

                dfsUtil(neighbor, visited);

            }
```

```
        }

    }

    public static void main(String[] args) {

        GraphDFS graph = new GraphDFS(7);

        graph.addEdge(0,1);
        graph.addEdge(0,2);
        graph.addEdge(0,3);
        graph.addEdge(1,4);
        graph.addEdge(1,5);
        graph.addEdge(3,6);

        graph.dfs(0);

    }

}
```

Output

```
0 1 4 5 2 3 6
```

Dry Run

Graph

Visit

```
0
```

Visited

```
0
```

Move to

```
1
```

Visited

```
0 1
```

Move to

```
4
```

Visited

```
0 1 4
```

No unvisited neighbors.

Backtrack.

Visit

```
5
```

Visited

```
0 1 4 5
```

Backtrack.

Return to

```
0
```

Visit

```
2
```

Visited

```
0 1 4 5 2
```

Backtrack.

Visit

```
3
```

Visited

```
0 1 4 5 2 3
```

Visit

```
6
```

Visited

```
0 1 4 5 2 3 6
```

Traversal Complete.

DFS Using an Explicit Stack

DFS can also be implemented iteratively.

Instead of recursion, we use a Stack.

Algorithm

1. Push starting vertex.
2. While stack is not empty:
    - Pop the top vertex.
    - If unvisited:
        - Visit it.
        - Push all unvisited neighbors.

Java Implementation (Iterative DFS)

```
import java.util.ArrayList;
import java.util.Stack;

public class IterativeDFS {

    private int vertices;
    private ArrayList<ArrayList<Integer>> adjList;

    public IterativeDFS(int vertices) {

        this.vertices = vertices;

        adjList = new ArrayList<>();

        for(int i=0;i<vertices;i++)
            adjList.add(new ArrayList<Integer>());

    }

    public void addEdge(int source,int destination){

        adjList.get(source).add(destination);
        adjList.get(destination).add(source);

    }

    public void dfs(int start){

        boolean visited[] = new boolean[vertices];
```

```
        Stack<Integer> stack = new Stack<>();

        stack.push(start);

        while(!stack.isEmpty()){

            int current = stack.pop();

            if(visited[current])
                continue;

            visited[current]=true;

            System.out.print(current+" ");

            for(int i=adjList.get(current).size()-1;i>=0;i--){

                int neighbor=adjList.get(current).get(i);

                if(!visited[neighbor])
                    stack.push(neighbor);

            }

        }

    }

    public static void main(String args[]){

        IterativeDFS graph = new IterativeDFS(7);

        graph.addEdge(0,1);
        graph.addEdge(0,2);
        graph.addEdge(0,3);
        graph.addEdge(1,4);
        graph.addEdge(1,5);
        graph.addEdge(3,6);

        graph.dfs(0);

    }

}
```

Output

```
0 1 4 5 2 3 6
```

DFS on a Disconnected Graph

Consider

```
0 ---- 1

2 ---- 3 ---- 4

5
```

Calling

```
dfs(0)
```

visits only

```
0 1
```

Vertices

```
2 3 4 5
```

remain unvisited.

Complete DFS Traversal

```
public void dfsTraversal() {

    boolean[] visited = new boolean[vertices];

    for (int i = 0; i < vertices; i++) {

        if (!visited[i]) {

            dfsUtil(i, visited);

        }
```

```
    }
}
```

Output

```
0 1 2 3 4 5
```

DFS Using an Adjacency Matrix

```
public void dfsMatrix(int[][] graph, int current, boolean[] visited) {

    visited[current] = true;

    System.out.print(current + " ");

    for (int i = 0; i < graph.length; i++) {

        if (graph[current][i] == 1 && !visited[i]) {

            dfsMatrix(graph, i, visited);

        }

    }

}
```

Recursive Call Stack Example

Graph

```
0
|
1
|
2
|
3
```

```
|
4
```

Call Stack

```
dfs(0)

↓

dfs(1)

↓

dfs(2)

↓

dfs(3)

↓

dfs(4)

↓

Return

↓

Return

↓

Return

↓

Return

↓

Return
```

Maximum recursion depth equals the longest path in the graph.

Time Complexity

Using an Adjacency List

Operation	Complexity
Visit Vertices	O(V)
Visit Edges	O(E)
Total	**O(V + E)**

Using an Adjacency Matrix

Every vertex scans an entire row.

Operation	Complexity
Total	**$O(V^2)$**

Space Complexity

Visited Array

```
O(V)
```

Recursive Call Stack

Worst Case

```
O(V)
```

Total

```
O(V)
```

BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal Style	Level by Level	Depth First
Shortest Path	Yes (Unweighted)	No
Memory Usage	More	Less (usually)
Backtracking	No	Yes
Typical Uses	Shortest Path, Level Traversal	Cycle Detection, Topological Sort, SCC

Applications of DFS

- Cycle Detection
- Topological Sorting
- Strongly Connected Components
- Finding Connected Components
- Maze Solving
- Path Finding
- Backtracking Problems
- Sudoku Solver
- N-Queens Problem
- Detecting Bridges
- Detecting Articulation Points

Common Mistakes

1. Forgetting the visited array.
2. Marking a vertex as visited after recursion instead of before.
3. Infinite recursion due to cycles.
4. Stack Overflow for very deep recursive graphs.
5. Assuming DFS always produces the same traversal order.
6. Forgetting to perform DFS from every unvisited vertex in disconnected graphs.

Interview Questions

1. Why is DFS naturally implemented using recursion?
2. Why do we need a visited array?
3. What is backtracking?
4. Can DFS find the shortest path?
5. Compare recursive and iterative DFS.
6. Why can recursive DFS cause a Stack Overflow?
7. Compare BFS and DFS with real-world examples.
8. Explain the time complexity of DFS.

Practice Questions

1. Implement recursive DFS.
2. Implement iterative DFS using a Stack.
3. Traverse a disconnected graph using DFS.
4. Count the number of connected components using DFS.
5. Print the DFS traversal order for a given graph.
6. Find whether a path exists between two vertices using DFS.
7. Modify DFS to store the parent of every visited vertex.
8. Print all vertices reachable from a given source.
9. Implement DFS using an adjacency matrix.
10. Compare the outputs of BFS and DFS for the same graph.

LeetCode and GeeksforGeeks Practice

Problem	Platform
Number of Islands (200)	LeetCode
Clone Graph (133)	LeetCode
Flood Fill (733)	LeetCode
Surrounded Regions (130)	LeetCode

Problem	Platform
Keys and Rooms (841)	LeetCode
All Paths From Source to Target (797)	LeetCode
Path With Maximum Gold (1219)	LeetCode
DFS of Graph	GeeksforGeeks
Connected Components	GeeksforGeeks
Graph Traversal	GeeksforGeeks

Key Points

- DFS explores one path completely before exploring another.
- DFS uses recursion or an explicit stack.
- Every vertex is visited exactly once.
- DFS runs in **O(V + E)** using an adjacency list.
- DFS is widely used in cycle detection, topological sorting, connected components, and many backtracking algorithms.
- The traversal order depends on the order in which neighbors are stored.